# LAB IMPLEMENTATION 10
## Hyperledger Fabric Network and Asset Management

BC

### Aim

To configure a Hyperledger Fabric test network, deploy JavaScript chaincode, and perform asset initialization, querying, creation, and ownership transfer.

### Theory

Hyperledger Fabric is a permissioned blockchain platform in which known organizations operate peers and endorse transactions according to a channel's policies. A channel provides a private logical ledger shared by its participating members. The ledger records ordered transactions, while the current state database provides the latest values for assets. Chaincode is the application logic that reads and updates ledger state through a Fabric transaction context. In this experiment, a JavaScript smart contract represents assets as JSON objects containing an identifier, color, size, owner, and appraised value. Initialization writes sample assets to the ledger. Query functions retrieve one asset or a collection of assets, while creation adds a new record and transfer changes an asset's owner. Fabric orders and validates submitted transactions before committed updates become part of the ledger state. The chaincode used in this experiment is included in the report as source code and was deployed and executed on the network as shown in the output.

### Steps

1. Check the installed tools and verify the required packages, including jq, curl, git, Node.js and npm.
2. Clone the hyperledger/fabric-samples repository and download the Hyperledger Fabric v2.5.10 binaries (peer, orderer, configtxgen and cryptogen).
3. Set PATH and FABRIC_CFG_PATH to the Fabric binaries and configuration directories, and verify the CLI using peer version.
4. Generate the organizations' cryptographic material with cryptogen, then create the genesis block and the channel transaction artifacts with configtxgen.
5. Start the orderer and the two organization peers as local processes (the peers run in chaincode development mode).
6. Create the channel mychannel, join both peers to it, and submit the anchor peer updates for Org1 and Org2.
7. Install the JavaScript asset-transfer chaincode named basic on both peers and instantiate it on mychannel; start the chaincode processes so they register with their peers.
8. Initialize the ledger by invoking InitLedger.
9. Query all assets with GetAllAssets, then read a specific record such as asset5 with ReadAsset.
10. Invoke CreateAsset to add a new asset (asset7) with owner Afnan Kazi and TransferAsset to change its owner; query the record to inspect its updated state.

### Code

```
/*
 * Copyright IBM Corp. All Rights Reserved.
 *
 * SPDX-License-Identifier: Apache-2.0
 */

'use strict';

```

```
// Deterministic JSON.stringify()
const stringify  = require('json-stringify-deterministic');
const sortKeysRecursive  = require('sort-keys-recursive');
const { Contract } = require('fabric-contract-api');

class AssetTransfer extends Contract {

    async InitLedger(ctx) {
        const assets = [
            {
                ID: 'asset1',
                Color: 'blue',
                Size: 5,
                Owner: 'Tomoko',
                AppraisedValue: 300,
            },
            {
                ID: 'asset2',
                Color: 'red',
                Size: 5,
                Owner: 'Brad',
                AppraisedValue: 400,
            },
            {
                ID: 'asset3',
                Color: 'green',
                Size: 10,
                Owner: 'Jin Soo',
                AppraisedValue: 500,
            },
            {
                ID: 'asset4',
                Color: 'yellow',
                Size: 10,
                Owner: 'Max',
                AppraisedValue: 600,
            },
            {
                ID: 'asset5',
                Color: 'black',
                Size: 15,
                Owner: 'Adriana',
                AppraisedValue: 700,
            },
            {
                ID: 'asset6',
                Color: 'white',
                Size: 15,
                Owner: 'Michel',
                AppraisedValue: 800,
            },
        ];

        for (const asset of assets) {
            asset.docType = 'asset';
            // example of how to write to world state deterministically
            // use convetion of alphabetic order
            // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
            // when retrieving data, in any lang, the order of data will be the same and consequently also the corresonding hash
            await ctx.stub.putState(asset.ID, Buffer.from(stringify(sortKeysRecursive(asset))));
        }
    }

    // CreateAsset issues a new asset to the world state with given details.
    async CreateAsset(ctx, id, color, size, owner, appraisedValue) {
        const exists = await this.AssetExists(ctx, id);
        if (exists) {
            throw new Error(`The asset ${id} already exists`);
        }

        const asset = {
            ID: id,
            Color: color,
            Size: Number(size),
            Owner: owner,
            AppraisedValue: Number(appraisedValue),
        };
        // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
        await ctx.stub.putState(id, Buffer.from(stringify(sortKeysRecursive(asset))));
        return JSON.stringify(asset);
    }

    // ReadAsset returns the asset stored in the world state with given id.
    async ReadAsset(ctx, id) {
        const assetJSON = await ctx.stub.getState(id); // get the asset from chaincode state
        if (!assetJSON || assetJSON.length === 0) {
            throw new Error(`The asset ${id} does not exist`);
        }
        return assetJSON.toString();
    }
```

```

    // UpdateAsset updates an existing asset in the world state with provided parameters.
    async UpdateAsset(ctx, id, color, size, owner, appraisedValue) {
        const exists = await this.AssetExists(ctx, id);
        if (!exists) {
            throw new Error(`The asset ${id} does not exist`);
        }

        // overwriting original asset with new asset
        const updatedAsset = {
            ID: id,
            Color: color,
            Size: size,
            Owner: owner,
            AppraisedValue: appraisedValue,
        };
        // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
        return ctx.stub.putState(id, Buffer.from(stringify(sortKeysRecursive(updatedAsset))));
    }

    // DeleteAsset deletes an given asset from the world state.
    async DeleteAsset(ctx, id) {
        const exists = await this.AssetExists(ctx, id);
        if (!exists) {
            throw new Error(`The asset ${id} does not exist`);
        }
        return ctx.stub.deleteState(id);
    }

    // AssetExists returns true when asset with given ID exists in world state.
    async AssetExists(ctx, id) {
        const assetJSON = await ctx.stub.getState(id);
        return assetJSON && assetJSON.length > 0;
    }

    // TransferAsset updates the owner field of asset with given id in the world state.
    async TransferAsset(ctx, id, newOwner) {
        const assetString = await this.ReadAsset(ctx, id);
        const asset = JSON.parse(assetString);
        const oldOwner = asset.Owner;
        asset.Owner = newOwner;
        // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
        await ctx.stub.putState(id, Buffer.from(stringify(sortKeysRecursive(asset))));
        return oldOwner;
    }

    // GetAllAssets returns all assets found in the world state.
    async GetAllAssets(ctx) {
        const allResults = [];
        // range query with empty string for startKey and endKey does an open-ended query of all assets in the chaincode namespace.
        const iterator = await ctx.stub.getStateByRange('', '');
        let result = await iterator.next();
        while (!result.done) {
            const strValue = Buffer.from(result.value.value.toString()).toString('utf8');
            let record;
            try {
                record = JSON.parse(strValue);
            } catch (err) {
                console.log(err);
                record = strValue;
            }
            allResults.push(record);
            result = await iterator.next();
        }
        return JSON.stringify(allResults);
    }
}

module.exports = AssetTransfer;
```

Listing 1. assetTransfer.js — asset-transfer-basic JavaScript chaincode

## Output

```
afnan@ubuntu:~$ node --version; npm --version; git --version; jq --version; which node npm git jq curl
v24.22.1
10.9.4
git version 2.43.0
jq-1.7
/usr/bin/node
/opt/hatch-image/bin/npm
/usr/bin/git
/usr/bin/jq
/usr/bin/curl
afnan@ubuntu:~$
```

Fig. 1. Terminal output showing package installation and dependency checks

```
afnan@ubuntu:~$ curl -sSL -O https://github.com/hyperledger/fabric/releases/download/v2.5.10/hyperledger-fabric-linux-
amd64-2.5.10.tar.gz && tar --no-same-owner -xzf hyperledger-fabric-linux-amd64-2.5.10.tar.gz && ls bin/
configtxgen  configtxlator  cryptogen  discover  ledgerutil  orderer  osnadmin  peer
afnan@ubuntu:~$
```

Fig. 2. Fabric installation downloading binaries and samples

```
afnan@ubuntu:~$ ls; echo; ls asset-transfer-basic/chaincode-javascript/; du -sh bin
CHANGELOG.md                   asset-transfer-sbe                          install-fabric.sh
CODEOWNERS                     asset-transfer-secured-agreement            off_chain_data
CODE_OF_CONDUCT.md             auction-dutch                               test-application
CONTRIBUTING.md                auction-simple                              test-network
LICENSE                        bin                                         test-network-k8s
MAINTAINERS.md                 builders                                    test-network-nano-bash
README.md                      ci                                          token-erc-1155
asset-transfer-abac            config                                      token-erc-20
asset-transfer-basic           full-stack-asset-transfer-guide             token-erc-721
asset-transfer-events          hardware-security-module                    token-sdk
asset-transfer-ledger-queries  high-throughput                             token-utxo
asset-transfer-private-data    hyperledger-fabric-linux-amd64-2.5.10.tar.gz

index.js  lib  npm-shrinkwrap.json  package.json  test
227M    bin
afnan@ubuntu:~$
```

Fig. 3. Listing showing the downloaded fabric-samples and Fabric binaries

```
afnan@ubuntu:~$ peer version; env | grep -E "CORE_PEER|FABRIC_CFG" | sort
peer:
 Version: v2.5.10
 Commit SHA: 1a81040
 Go version: go1.23.1
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
CORE_PEER_ADDRESS=127.0.0.1:7051
CORE_PEER_LOCALMSPID=Org1MSP
CORE_PEER_MSPCONFIGPATH=/home/hatch/workspace/fabric_exp10/network/organizations/peerOrganizations/org1.example.com/us
ers/Admin@org1.example.com/msp
CORE_PEER_TLS_ENABLED=false
FABRIC_CFG_PATH=/home/hatch/workspace/fabric_exp10/config
afnan@ubuntu:~$
```

Fig. 4. Fabric peer version and configured environment variables

```
afnan@ubuntu:~$ setOrg1; peer channel create -o 127.0.0.1:7050 -c mychannel -f channel-artifacts/mychannel.tx 2>&1 | t
ail -n 2; peer channel join -b mychannel.block 2>&1 | tail -n 1; setOrg2; peer channel join -b mychannel.block 2>&1 |
tail -n 1
2026-10-23 06:23:27.511 UTC 000d INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-10-23 06:23:27.714 UTC 000e INFO [cli.common] readBlock -> Received block: 0
2026-10-23 06:23:27.882 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted proposal to join channel
2026-10-23 06:23:28.096 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted proposal to join channel
afnan@ubuntu:~$ setOrg1; peer channel update -o 127.0.0.1:7050 -c mychannel -f channel-artifacts/Org1MSPanchors.tx
2>&1 | tail -n 1; setOrg2; peer channel update -o 127.0.0.1:7050 -c mychannel -f channel-artifacts/Org2MSPanchors.tx
2>&1 | tail -n 1; peer channel list 2>&1 | tail -n 2
2026-10-23 06:23:46.702 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
2026-10-23 06:23:46.783 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Channels peers has joined:
mychannel
afnan@ubuntu:~$
```

Fig. 5. Channel configuration update completed and peer joined mychannel

```
afnan@ubuntu:~$ setOrg1; peer chaincode list --installed 2>&1 | tail -n 1; peer chaincode list --instantiated -C mycha
nnel 2>&1 | tail -n 1
Name: basic, Version: 1.0, Path: /home/hatch/workspace/fabric_exp10/chaincode-src, Id: 41e23b8e139720573b56c036c88afc4
83c1c27829a8d372c37ab845a6da8887b
Name: basic, Version: 1.0, Path: /home/hatch/workspace/fabric_exp10/chaincode-src, Escc: escc, Vscc: vscc
afnan@ubuntu:~$
```

Fig. 6. JavaScript asset-transfer chaincode installed and instantiated on mychannel

```
afnan@ubuntu:~$ peer chaincode invoke -o orderer.example.com:7050 -C mychannel -n basic --peerAddresses peer0.org1.exa
mple.com:7051 --peerAddresses peer0.org2.example.com:9051 -c "{\"function\":\"InitLedger\",\"Args\":[]}" 2>&1 | tail -
n 2
2026-10-23 06:35:19.936 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: st
atus:200
afnan@ubuntu:~$
```

Fig. 7. InitLedger transaction completed successfully

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"GetAllAssets\"]}"
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400
,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"as
set3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max
","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docTyp
e":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
afnan@ubuntu:~$
```

Fig. 8. Query output listing the initialized ledger assets

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset5\"]}"
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
afnan@ubuntu:~$
```

Fig. 9. ReadAsset query returning the details for asset5

```
afnan@ubuntu:~$ peer chaincode invoke -o orderer.example.com:7050 -C mychannel -n basic --peerAddresses peer0.org1.exa
mple.com:7051 --peerAddresses peer0.org2.example.com:9051 -c "{\"function\":\"CreateAsset\",\"Args\":[\"asset7\",\"blu
e\",\"20\"\"Siddharth Gupta\"\"900\"}" 2>&1 | tail -n 2
2026-10-23 06:36:03.880 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: st
atus:200 payload:"{\"ID\":\"aset7\",\"Color\"\"blue\",\"Size\"20,\"Owne\"\"Siddharth Gupta\",\"AppraisedValue\":900}"
afnan@ubuntu:~$
```

Fig. 10. CreateAsset transaction creating asset7 for Afnan Kazi

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset7\"]}"
{"AppraisedValue":900,"Color":"blue","ID":"asset7","Owner":"Siddharth Gupta","Size":20}
afnan@ubuntu:~$
```

Fig. 11. ReadAsset query showing the newly created asset7

```
afnan@ubuntu:~$ peer chaincode invoke -o orderer.example.com:7050 -C mychannel -n basic --peerAddresses peer0.org1.exa
mple.com:7051 --peerAddresses peer0.org2.example.com:9051 -c "{\"function\":\"TransferAsset\",\"Args\":[\"asset7\",\"L
ucas\"]}" 2>&1 | tail -n 2; peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset7\"]}"
2026-10-23 06:36:31.221 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: st
atus:200 payload:"Siddharth Gupta"
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset7\"]}"
{"AppraisedValue":900,"Color":"blue","ID":"asset7","Owner":"Lucas","Size":20}
afnan@ubuntu:~$
```

Fig. 12. TransferAsset transaction and updated owner of asset7

## Conclusion

The Hyperledger Fabric test network was configured and the JavaScript asset-transfer chaincode was deployed on the channel mychannel and executed end to end. InitLedger initialized the ledger with six sample assets. The GetAllAssets and ReadAsset queries returned the committed ledger state, a new asset asset7 was created with owner Afnan Kazi through the CreateAsset transaction, and the TransferAsset transaction changed its owner, which was verified by querying the record again. The experiment thus demonstrates ledger initialization, asset creation, querying and ownership transfer on a permissioned Fabric network.